

CSE 727 · REINFORCEMENT LEARNING · SPRING 2026

HOMEWORK 2

NSYSU Drone Control

with Reinforcement Learning

Department of Computer Science and Engineering · National Sun Yat-sen University

DUE

May 30, 2026

POINTS

100 (+5)

FORMAT

Individual  github.com/NSYSU-ARL/Assignment2

ASSIGNMENT 2

At a glance

100

+5 bonus

Total Points

2

parts

Environment + RL

May 30

23:59

Due Date (Sat)

4-8

pages PDF

Reflection Report



Your Mission

Use ROS 2 + Gazebo to formulate drone control as an MDP, train an RL agent, and write a reflective report comparing it to classical control.

Learning objectives



ROS 2 Workflow

Get comfortable with publisher/subscriber patterns and the Gazebo physics simulation loop.



PID vs. RL Control

Compare classical control against learned policies in continuous action spaces.



MDP Formulation

Express the drone task as (S, A, R, γ) and design an informative reward.



Train & Analyze

Train a modern RL algorithm (PPO / SAC / TD3) and read training curves critically.



Communicate

Write a formal report with motivation, literature review, and honest failure analysis.

The NSYSU Drone environment



A ROS 2 port of tum_simulator

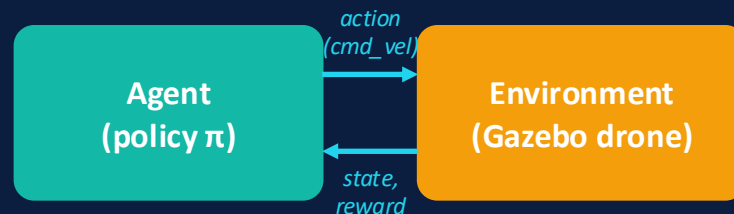
Includes:

- Quadrotor model
- plugin_drone (built-in PID)
- Sensors: IMU, GPS, sonar, camera

Recommended:

- Ubuntu 22.04 · Docker 24.x
- ROS 2 Iron (Humble / Rolling also OK)
- Headless gzserver if no NVIDIA GPU

The RL loop



Gazebo Classic handles the physics; your code only sees ROS 2 topics.

ROS 2 Iron

Gazebo Classic

Gymnasium

SB3 / PyTorch

Key ROS 2 topics you'll use

Topic	Message type	Purpose
<code>/simple_drone/cmd_vel</code>	<code>geometry_msgs/Twist</code>	Velocity command — your ACTION output
<code>/simple_drone/gt_pose</code>	<code>geometry_msgs/Pose</code>	Ground-truth pose — main OBSERVATION
<code>/simple_drone/gt_vel</code>	<code>geometry_msgs/Twist</code>	Ground-truth velocity
<code>/simple_drone/takeoff</code>	<code>std_msgs/Empty</code>	Take off (required after each reset)
<code>/simple_drone/reset</code>	<code>std_msgs/Empty</code>	Reset drone pose (end of episode)
<code>/simple_drone/sonar/out</code>	<code>sensor_msgs/Range</code>	Downward range sensor (obstacle cues)



Always publish takeoff after each reset, otherwise the drone stays on the ground and no RL episode will make sense.

Two parts, one full pipeline

PART 1

Environment Setup & fly_straight.py

20 pts

- Build the Docker image & run the VNC viewer
- Launch Gazebo + RViz with launch_drone
- Publish takeoff, then run fly_straight.py
- Test ≥ 3 alternative target points
- Describe effect of K_p and max_speed (Appendix A)

PART 2

Applying RL Algorithms

75 pts

- Pick a task (A–E) or propose your own
- Wrap environment as gymnasium.Env
- Train PPO / SAC / TD3 / DDPG (or justified custom)
- Design & justify the reward function
- Produce training curves + test script
- Write a 4–8 page reflection report (PDF)

Copying rl_fly_to_target.py with no meaningful extension will not receive credit.

Get the simulator running

1

Build & launch

```
$ docker build -t nsysu_drone_vnc:iron
```

Then `./run_docker.sh` to start the container.

2

Connect via VNC

```
$ open http://localhost:5901
```

Password: `nsysudrone` · XFCE desktop appears.

3

Launch the drone

```
$ launch_drone
```

Gazebo + RViz open; confirm the quadrotor is visible.

4

Take off & fly

```
$ python3 fly_straight.py
```

Flies to default target (5.0, 3.0, 2.0).

DELIVERABLES

Screenshot(s) of Gazebo · Terminal log of a successful run · ½-page write-up on K_p & `max_speed` (→ Appendix A)

Exploring fly_straight.py



What to try

Change the target point at least 3 times:

- $(-2, 4, 1.5)$
- $(0, 0, 3)$
- $(6, -3, 2)$

Record the flight behavior for each case.

Then tweak K_p and max_speed and note what changes.



Evidence checklist

- ✓ Gazebo screenshot(s) clearly showing drone and target
- ✓ Terminal log of a successful fly_straight.py run
- ✓ Observations table for each of your 3+ targets
- ✓ ½-page write-up on increasing/decreasing K_p
- ✓ ½-page write-up on adjusting max_speed

Minimum requirements



Gym interface

Subclass `gym.Env`. Implement `reset` / `step` and define `observation_space` and `action_space`.



RL algorithm

Use ≥ 1 algorithm (PPO / SAC / TD3 / DDPG). Custom algorithms must be explained in the report.



Justified reward

Explain the meaning and weight of every term in your reward function.



Explicit termination

Define episode-end conditions: target reached, timeout, crash, out of bounds.



Training curves

Save reward vs. episode/timestep as a figure that is embedded in the report.



Test script for TA

Loads your trained model and demonstrates agent behavior in Gazebo out-of-the-box.

Pick one — or pitch your own

TASK A	TASK B	TASK C	TASK D	TASK E
				
Precision Hovering	Random Target Navigation	Trajectory Tracking	Obstacle Avoidance	Multi-Waypoint Cruising
Hover at (x, y, z) and stay stable under motionDriftNoise.	New target each episode — learn a policy that generalizes.	Follow a time-varying reference (figure-eight, circle, helix).	Add obstacles; use sonar/camera to navigate around them.	Visit an ordered waypoint list, minimizing total time.



Submissions that are a direct copy of `rl_fly_to_target.py` with no meaningful extension will not receive credit.

Five sections · 4–8 pages · PDF

1

Task Definition & Motivation

Describe the task you want the agent to learn and argue why it's worth studying.

5 pts

2

Pain Points of Existing Methods

Show concretely where PID / LQR / MPC / hand-crafted rules struggle — with examples.

5 pts

3

Literature Review

≥ 3 papers (last 5 years, PPO/SAC/TD3/DDPG or UAV RL). Say how each inspired your design.

8 pts

4

Proposed Solution

Algorithm choice, MDP (S, A, R, γ), network, hyperparameters, and reward rationale.

7 pts

5

Results & Discussion

Training curves, success rate, failure cases, and comparison to the P-controller baseline.

5 pts

Writing tips: Be specific about PID's weaknesses · Explain how each paper shaped your design · Honest failure analysis > cherry-picked curves

Package everything in one ZIP



WHAT'S INSIDE



Source code

Gym env + train + test scripts



Trained model

.zip (SB3) or .pth (PyTorch)



Report PDF

HW2_StudentID_report.pdf



Part 1 artifacts

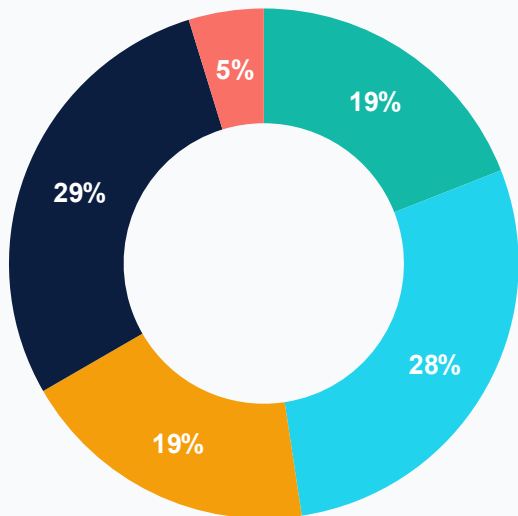
Screenshots + terminal log



README.md

Env, install, reproduce, test

How your 100 points are allocated



■ Part 1 ■ Part 2 - Code ■ Part 2 - Model ■ Part 2 - Report ■ Bonus

Part 1

Setup + fly_straight.py + logs/screenshots

+20**Part 2 - Code**

Gym env, reward design, training pipeline

+30**Part 2 - Model**

Loads cleanly & hits target success rate

+20**Part 2 - Report**

5 sections (see Report slide) + presentation

+30**Bonus**

Baselines · extra task · sim-to-real discussion

+5**TOTAL****100 (+5)**

How each deliverable is scored

Part 1	20 pts	Part 2 · Code	30 pts	Part 2 · Model	20 pts
10	Environment set up and demonstrated	8	Gym env (reset/step/spaces) is correct	5	Loads cleanly — no version conflicts
6	≥ 3 alternative targets tested & documented	6	Reward design well reasoned	15	Reasonable success rate on TA test scenario
4	Written notes on Kp / max_speed (Appendix A)	6	Training script + checkpointing + logging		
		5	Test script loads model & runs		
		5	Code readability & comments		

Report rubric (30 pts) is itemized on the Report slide. Presentation quality may adjust ±2 pts.

Academic integrity & AI usage

DON'T

- Copy another student's code or report → both parties get 0
- Paste paragraphs from published papers (= plagiarism)
- Submit a report that is entirely AI-generated
- Copy figures from the internet as your own
- Re-submit `rl_fly_to_target.py` as your Part 2

MUST

- Write your own code and your own report
- Cite papers accurately (IEEE or APA)
- Produce your own figures with numbers & captions
- Discussion with classmates is allowed
- Disclose AI tool usage in an Acknowledgement section

Where to learn more & get help



NSYSU Drone (README & plugins)

Start here — plugin_drone & pid_controller source

github.com/NSYSU-ARL/Assignment2



ROS 2 Iron docs

Topics, nodes, launch files

docs.ros.org/en/iron/



Gazebo Classic tutorials

World files, plugins, sensors

classic.gazebosim.org/tutorials



Stable-Baselines3

PPO / SAC / TD3 / DDPG implementations

stable-baselines3.readthedocs.io



Gymnasium

Env interface: reset / step / spaces

gymnasium.farama.org



Troubleshooting

Check the Troubleshooting section of README.md

[README.md](#) · [Troubleshooting](#)

GOOD LUCK

May your reward
curves
only go up. ↗

Any Questions?



DEADLINE

May 30

2026 · Saturday · 23:59

No delays allowed

Upload ZIP to eLearn following the folder structure.